

Multimodal Incremental RAG for Domain-Specific Assistants Design, Cost-Aware Updating, and Empirical Evaluation

HUPEDCARE Project Team

April 12, 2026

Abstract

This manuscript presents a scientific framing for the HUPEDCARE chatbot system, a Retrieval-Augmented Generation (RAG) architecture that combines heterogeneous data ingestion (FTP files, SQL records, text documents, images, and audio), incremental context rebuilding, and vector retrieval for grounded question answering. The paper is structured using an IMRaD-style organization and includes a reproducible experimental protocol focused on quality, latency, and cost.

1 Introduction

Domain-specific assistants require factual consistency, source grounding, and operational efficiency. In practical deployments, knowledge sources are heterogeneous and evolve over time, introducing both retrieval complexity and recurring indexing costs. The HUPEDCARE project addresses this challenge through a multimodal RAG pipeline that normalizes content into a unified master context and updates vector embeddings only when relevant changes are detected.

This paper is guided by the following research questions:

- RQ1: Does multimodal RAG improve factual answer quality over non-retrieval and text-only baselines?
- RQ2: Does incremental hash-based updating reduce processing cost and indexing latency without degrading answer quality?
- RQ3: What is the marginal impact of each source family (SQL, web pages, documents, images, audio)?
- RQ4: How does chunking granularity affect retrieval recall, end-to-end latency, and final response quality?

2 Related Work

Retrieval-Augmented Generation (RAG) has become a practical pattern for reducing hallucinations and improving factual grounding in large language model systems. Early formulations combine dense retrieval with sequence generation [5], while subsequent systems emphasize modular retrieval stacks, orchestration, and production constraints [3, 8]. The present project follows this line by explicitly separating indexing and serving concerns and by enforcing source-aware prompt construction.

Our retrieval layer relies on vector similarity over chunked context representations, consistent with dense retrieval paradigms such as DPR [4]. The effectiveness of this design is closely related to embedding quality and chunk construction. Recent embedding models have improved semantic retrieval quality in practical deployments [7]. In addition, chunk size and overlap are known to trade off recall, latency, and context coherence, motivating the parameter-sensitivity analysis proposed in this paper.

The multimodal component (images and audio alongside textual/SQL sources) is aligned with the broader trend toward unified multimodal models and pipelines [6]. In domain environments, however, multimodal extraction introduces additional noise channels (OCR errors, transcription variance), making robust normalization and provenance tagging essential. Our method addresses this by converting all sources into a single master context with explicit source delimiters.

From an operational perspective, cost-aware update strategies are increasingly relevant as embedding and generation APIs dominate recurrent expenditure. Incremental computation based on content hashes has long been a standard systems optimization and is naturally applicable to RAG indexing pipelines: unchanged corpora should not trigger full re-embedding. This paper contributes an empirical evaluation of that policy under realistic update regimes.

Evaluation methodology follows QA and retrieval best practices, combining quality metrics (factual correctness, grounding) with retrieval indicators (Recall@ k , MRR) and systems metrics (latency/cost). The emphasis on multi-objective analysis is consistent with emerging guidance for robust LLM system evaluation [1, 2].

3 Method

3.1 System Overview

The proposed architecture consists of two offline/online stages:

1. **Offline ingestion and indexing stage:** source synchronization, extraction, context assembly, and vector database update.
2. **Online serving stage:** retrieval and generation through a FastAPI endpoint that combines query-time retrieved context with a constrained system prompt.

The implementation is organized as follows:

- `src/collector.py`: orchestration of ingestion and indexing.
- `src/utils/helper.py`: extraction utilities and metadata persistence.
- `src/utils/sql_collector.py`: SQL-based structured content export.
- `src/utils/ftp_collector.py`: FTP synchronization with timestamp-aware mirroring.
- `src/utils/vector_processor.py`: context chunking and ChromaDB indexing.
- `src/server.py`: query-time retrieval and answer generation.

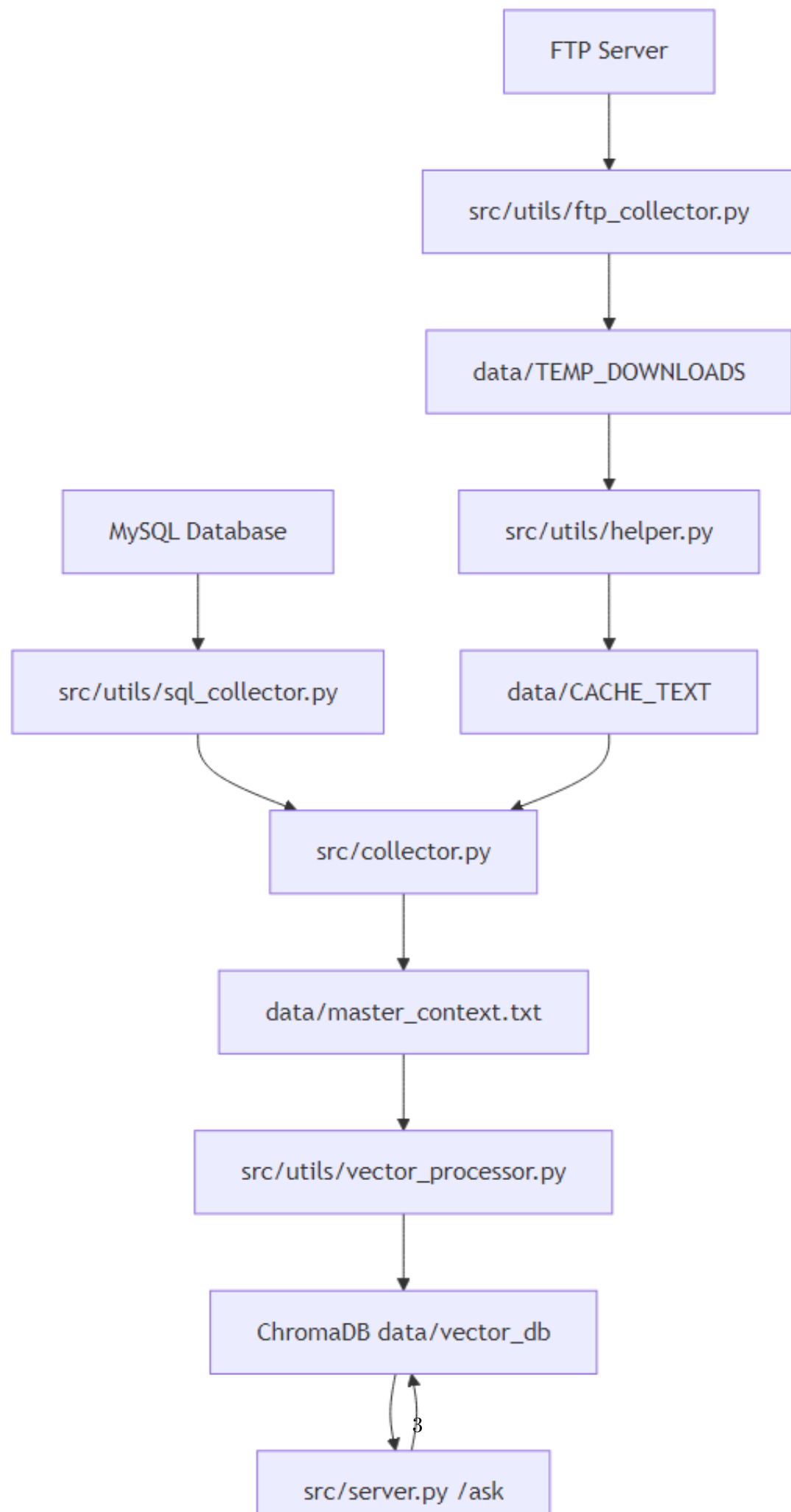


Figure 1 summarizes the full data and inference lifecycle implemented in this project.

3.2 Data Ingestion and Normalization

The ingestion pipeline processes heterogeneous sources into a unified textual representation:

- **File sources:** HTML/PHP, TXT, PDF, DOCX, legacy DOC, image files, and audio files.
- **Database sources:** SQL queries configured in `config/config.yaml` and executed against MySQL.

For each source type, extraction functions normalize content into plain text, remove boilerplate/noise, and produce cache-ready records. All cache records are merged into `data/master_context.txt` with explicit source headers to support response grounding.

3.3 Incremental Update Policy

To avoid unnecessary embedding recomputation, the pipeline computes a hash of `master_context.txt` and stores it in metadata. Let H_t be the hash at run t and H_{t-1} be the previously stored hash. Reindexing is triggered only if:

$$H_t \neq H_{t-1}. \quad (1)$$

This policy reduces API usage and runtime when source updates are sparse while preserving retrieval freshness after meaningful context changes.

3.4 Chunking and Vector Index Construction

The context text is segmented using a recursive splitter with configurable chunk size and overlap. Source delimiters (for example, source headers and SQL record markers) are prioritized as separators so semantically related snippets remain grouped. Chunks are embedded with the configured embedding model and inserted into a ChromaDB collection (`rag_context`).

3.5 Online Retrieval and Generation

At inference time, the API receives a user question, retrieves top- k chunks from the vector store, and injects them into the system prompt context block. The answer is generated by the configured chat model with constrained prompting that emphasizes source-aligned responses.

Formally, for query q , retriever R returns:

$$C_q = R(q, k) = \{c_1, c_2, \dots, c_k\}, \quad (2)$$

and the generator produces:

$$y = G(q, C_q, p_{sys}), \quad (3)$$

where p_{sys} is the system instruction template from configuration.

3.6 Implementation and Reproducibility Considerations

The system behavior is controlled through `config/config.yaml` and environment variables. Logging is centralized through rotating file handlers and console output, enabling traceability for ingestion, retrieval, and serving. Experimental reproducibility requires fixed model versions, fixed retrieval parameters, and a frozen test question set.

4 Experimental Design

4.1 Experimental Goals

The experimental campaign is designed to evaluate:

1. answer quality and grounding,
2. system latency and throughput,
3. indexing and inference cost,
4. robustness under varying update and source-ablation conditions.

4.2 Research Variables

Independent variables

- Retrieval mode: no-RAG, text-only RAG, multimodal RAG.
- Update mode: full rebuild vs incremental hash-based update.
- Source configuration: all sources vs ablation of one source family.
- Chunking parameters: chunk size and overlap grid.

Dependent variables

- Quality: factual correctness, semantic F1, grounding rate, hallucination rate.
- Retrieval: Recall@ k , Mean Reciprocal Rank (MRR).
- Systems: ingestion time, indexing time, end-to-end latency (p50/p95), throughput.
- Cost: number of embedded chunks, model-call count, estimated API cost per run and per 100 queries.

4.3 Benchmark Construction

A domain benchmark should be built with at least 150–300 questions, stratified by:

- direct factual lookup,
- cross-source aggregation,
- ambiguity resolution,
- out-of-context queries.

Each item should include expected answer criteria and, when possible, expected source attribution. The benchmark should be split into development and test subsets and evaluated over repeated runs.

4.4 Baselines and Ablations

- **Baseline A:** direct chat model without retrieval.
- **Baseline B:** RAG with text-only corpus.
- **System C:** full multimodal incremental RAG.

Ablation studies remove one source family at a time (for example SQL, images, audio) to estimate marginal contribution to quality and grounding.

4.5 Protocol

1. Freeze configuration and model versions.
2. Build or refresh context according to the update mode.
3. Execute the full benchmark on each configuration.
4. Repeat each configuration at least three times to capture variance.
5. Aggregate metrics and compute confidence intervals.

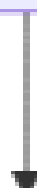
Define RQs and hypotheses



Build benchmark question
set



Freeze config and model
versions



Run ingestion and indexing
mode



Execute all⁷ baseline
systems

Figure 2 defines the evaluation pipeline used to ensure consistent comparisons across all baselines and ablations.

4.6 Statistical Analysis

For paired correctness outcomes across systems, McNemar testing can be applied. For aggregate metric differences, bootstrap confidence intervals (95%) are recommended. Along with p-values, effect sizes should be reported to quantify practical significance.

4.7 Threats to Validity

- **Internal validity:** extraction noise from OCR/transcription and source parsing errors.
- **Construct validity:** mismatch between evaluation rubric and real user utility.
- **External validity:** domain-specific conclusions may not transfer directly to other verticals.
- **Temporal validity:** model or API updates can shift observed performance over time.

5 Results

This section is organized to answer each research question with quantitative evidence. Replace placeholders with measured values from your benchmark runs.

5.1 RQ1: Quality and Grounding Across Baselines

Table 1: Answer quality and grounding by baseline configuration.

Configuration	Exact Match	Semantic F1	Grounding Rate	Hallucination Rate
No-RAG baseline	–	–	–	–
Text-only RAG	–	–	–	–
Multimodal incremental RAG	–	–	–	–

Interpretation template. The multimodal configuration improved factual quality by $X\%$ over the no-RAG baseline and by $Y\%$ over text-only RAG, while reducing hallucination rate by Z points.

5.2 RQ2: Cost and Latency Impact of Incremental Updates

Table 2: Update-mode comparison for indexing efficiency and serving behavior.

Update mode	Re-embedded chunks	Indexing time (s)	API cost / run	p95 answer latency (m)
Full rebuild	–	–	–	–
Incremental hash-based	–	–	–	–

Interpretation template. Hash-based updates reduced embedding calls by $X\%$ and indexing time by $Y\%$ without statistically significant quality loss.

5.3 RQ3: Source Ablation Analysis

Table 3: Marginal contribution of each source family.

Configuration	Exact Match	Grounding Rate	Delta vs full system
Full system (all sources)	–	–	0.00
Without SQL	–	–	–
Without image descriptions	–	–	–
Without audio transcriptions	–	–	–
Without file-based documents	–	–	–

Interpretation template. The largest degradation appeared when removing *source S*, indicating that this source category contributes most to domain factuality.

5.4 RQ4: Chunking Sensitivity

Table 4: Chunk-size and overlap sensitivity analysis.

Chunk size	Overlap	Recall@ <i>k</i>	MRR	Exact Match	p95 latency (ms)
Small / low overlap	–	–	–	–	–
Medium / medium overlap	–	–	–	–	–
Large / high overlap	–	–	–	–	–

Interpretation template. Medium chunk settings provided the best quality-latency compromise, with higher retrieval quality than large chunks and lower latency than very fine chunking.

5.5 Statistical Significance Summary

Table 5: Significance tests for key pairwise comparisons.

Comparison	Test	p-value	Effect size
Multimodal RAG vs no-RAG	McNemar / bootstrap	–	–
Multimodal RAG vs text-only RAG	McNemar / bootstrap	–	–
Incremental vs full rebuild	Bootstrap CI	–	–

5.6 Error Analysis

Report qualitative failure modes with representative examples, such as source mismatch, stale context, OCR/transcription artifacts, and retrieval misses. A compact taxonomy table is recommended for reproducibility.

6 Discussion

Interpret the empirical findings in terms of trade-offs between quality, cost, and operational complexity. Highlight where multimodality provides substantial gains and where simpler pipelines may remain competitive.

7 Limitations and Future Work

Candidate future directions include: confidence calibration, adaptive retrieval depth, temporal decay weighting for stale content, automated benchmark expansion, and stronger attribution verification.

8 Conclusion

This paper template describes a reproducible route for evaluating a practical multimodal incremental RAG assistant. The proposed methodology enables rigorous analysis of factual quality, grounding behavior, and cost-aware update strategies in real deployment settings.

Reproducibility Checklist

- Source code version and commit hash.
- Exact `config.yaml` used for experiments (excluding secrets).
- Model and embedding version identifiers.
- Benchmark question set and annotation protocol.
- Metric definitions and evaluation scripts.

Submission Readiness Checklist

- Related Work includes up-to-date references and explicit positioning.
- All claims in Results are backed by a table, figure, or statistical test.
- Experimental protocol includes benchmark split, repetitions, and fixed settings.
- Figures are exported and present in `docs/images/`.
- Bibliography compiles without missing entries.
- Language and formatting match target venue guidelines.
- Limitations and ethical considerations are explicitly discussed.

References

- [1] Yuzhong Chang, Xu Wang, Jindong Wang, Yuan Wu, Kaijie Zhu, Hao Chen, Linyi Yi, Xiaoyuan Wang, Wei Ye, Yue Zhang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 2024.
- [2] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217*, 2024.
- [3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kang Jia, Jinliang Pan, Yuxiang Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

- [4] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuettler, Mike Lewis, Wen-tau Yih, Tim Rocktaeschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [6] OpenAI. Gpt-4v(ision) system card. *arXiv preprint arXiv:2309.17421*, 2023.
- [7] OpenAI. New embedding models and api updates. OpenAI Blog, 2024. <https://openai.com/index/new-embedding-models-and-api-updates/>.
- [8] Shamane Siriwardhana, Ruvan Weerasinghe, Tanuja Kaluarachchi, Hyunwoo Byun, Jihyun Lee, Sung-Hyon Yoon, Hyeongu Lee, Surangika Lee, Kichang Nam, and Jinheon Lee. Improving retrieval-augmented large language models with iterative retrieval-generation synergy. *arXiv preprint arXiv:2305.15294*, 2023.